DATABASES

Schema Migrations Visual Guide

Expand/contract, zero-downtime deploys -- change the database without stopping the world.

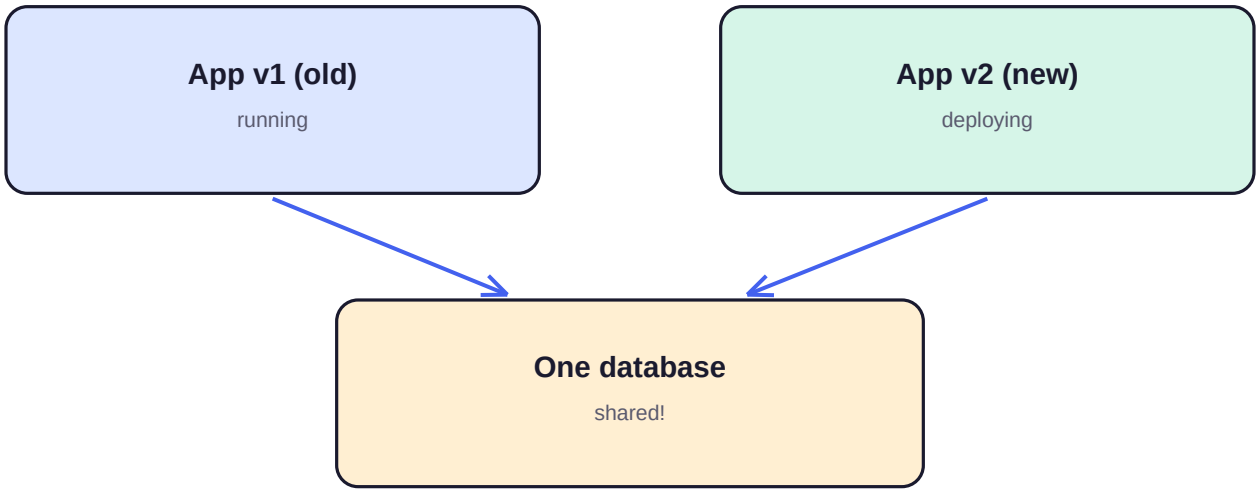
ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

The Zero-Downtime Problem

Old code and new schema must coexist during a deploy.

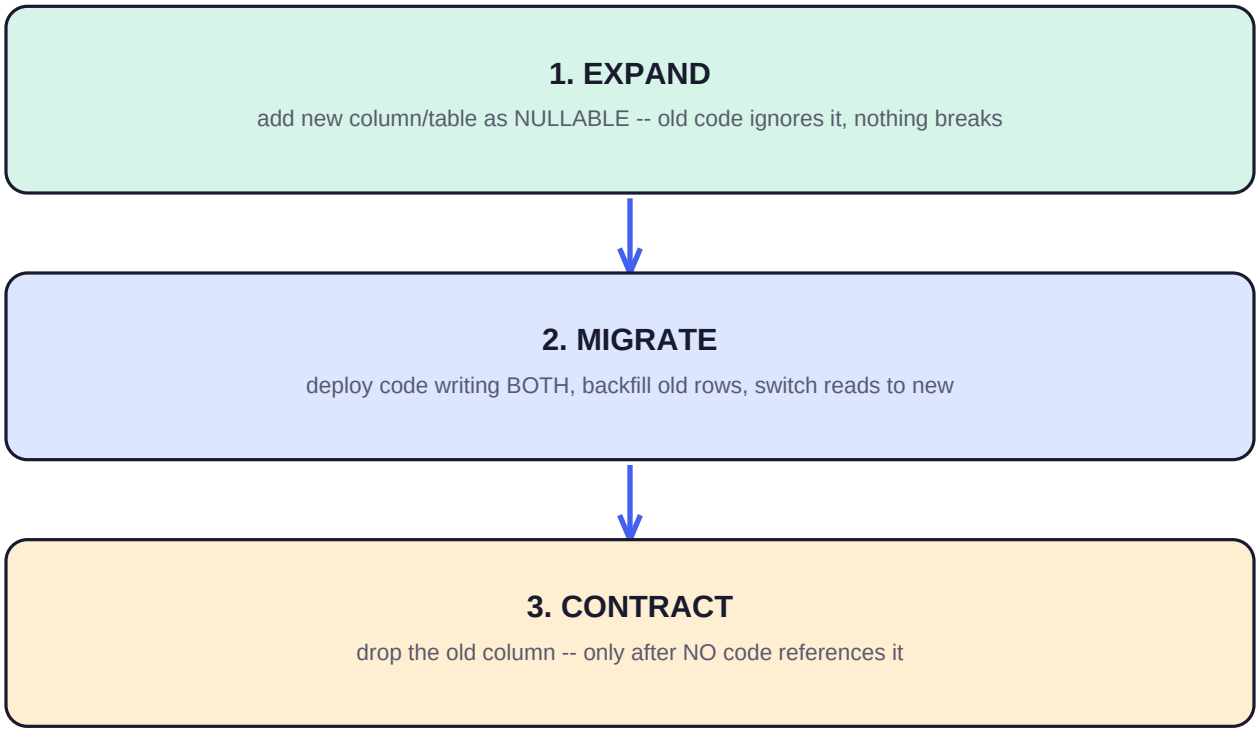


During rolling deploys both versions hit the same DB. Every migration must be compatible with BOTH.

STEP 2/6

The Expand/Contract Pattern

Three phases, zero breakage.



STEP 3/6

Backfill Strategies

Filling the new column on millions of rows.

Batched updates

UPDATE ... WHERE id BETWEEN x AND y LIMIT 10000 -- small txns, no lock storms

Throttled jobs

pause between batches; watch replica lag

Idempotent + resumable

track progress; crashes resume, never restart from zero

Never backfill in one giant transaction -- it locks the table and bloats the WAL.

STEP 4/6

Dangerous Operations

These lock, rewrite, or break -- handle with care.

Rename column

breaks old code instantly -- use expand/contract instead

ADD COLUMN ... NOT NULL

rewrites the table without a default -- add nullable, backfill, then constrain

DROP COLUMN / TABLE

irreversible without a restore -- contract phase only, after code is clean

DDL takes strong locks. On big tables, even 'fast' ALTERs deserve a maintenance window or online tooling.

STEP 5/6

Migration Tools

Versioned, reviewable, reversible.

Flyway / Liquibase

SQL-first versioned migrations -- the enterprise standard

Alembic

SQLAlchemy's companion -- autogenerate diffs, review them

Prisma / Django

ORM-managed -- great until you need raw SQL escape hatches

- Every migration: versioned in git, reviewed like code, runnable forward AND back.
- CI runs migrations against a scratch DB -- broken SQL never reaches production.

STEP ✓/✓

Key Takeaways

Schema Migrations -- Remember

- Rolling deploys mean old and new code share one DB -- every migration must suit both.
- Expand/contract: add nullable, dual-write + backfill, then drop the old -- never rename in place.
- Backfill in small throttled batches; make it idempotent and resumable.
- Renames, NOT NULL without defaults, and DROPs are the dangerous trio -- plan them explicitly.
- Version migrations in git, review them, test them in CI, keep them reversible.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev